



Resumo Abrangente - Orientação a Objetos

(Francisco Schneider)



AVISO DE CONFIDENCIALIDADE

ESTE MATERIAL FOI ELABORADO EXCLUSIVAMENTE PARA OS PARTICIPANTES DO WORKSHOP DE C# DO JEITO CERTO, CONDUZIDA PELOS PROFISSIONAIS DA EXIMIA.CO.

Os conteúdos aqui apresentados refletem discussões realizadas em grupo, incluindo contribuições diretas dos participantes. Portanto, não representam necessariamente a opinião dos instrutores ou de sua empresa. A distribuição, reprodução ou divulgação deste material fora do grupo de participantes do workshop, parcial ou integralmente, não é permitida sem autorização prévia expressa.

Pedimos respeito à confidencialidade do conteúdo e ao ambiente seguro de troca construído ao longo do Workshop de C# do Jeito Certo.

ABSTRACT

Francisco Schneider explora a Orientação a Objetos avançada no .NET, focando no design intencional. O ponto central é a integridade do Domínio, que não deve ser "simpático", mas rigoroso, atuando como o último cadeado de segurança para garantir o estado sempre válido e consistente dos objetos.

A aula combate a "Obsessão por Primitivos" através de Value Objects, utilizando recursos modernos como record structs. Schneider demonstra como o uso correto de Generics e operadores implícitos otimiza a performance, evita o boxing e facilita a modernização de sistemas legados de forma segura.

No tratamento de falhas, o instrutor compara Exceptions e Result Pattern, defendendo assinaturas de métodos mais "honestas". Ele detalha a preservação da Stack Trace e o uso do padrão Maybe para enriquecer a semântica, reservando exceções apenas para cenários verdadeiramente catastróficos.

Por fim, os princípios SOLID são abordados sob uma ótica prática de manutenção. Schneider destaca a importância da maturidade do time nas decisões arquiteturais, propondo interfaces como papéis específicos e a redução do impacto de mudanças para elevar a longevidade do software produzido.

SUMÁRIO

1. ENCAPSULAMENTO E A INTEGRIDADE DO DOMÍNIO	5
2. TIPAGEM SEMÂNTICA VS. OBSESSÃO POR PRIMITIVOS	5
3. HERANÇA, COMPOSIÇÃO E POLIMORFISMO	5
4. GENERICS: PERFORMANCE E ABSTRAÇÃO	6
5. TRATAMENTO DE FALHAS: EXCEPTIONS VS. RESULT PATTERN	6
6. SOLID SOB UMA ÓTICA PRÁTICA	6
CONCLUSÃO	7
INSIGHTS E TAKEAWAYS PRÁTICOS	8
1. FILOSOFIA DE DESIGN DE DOMÍNIO	8
2. MODELAGEM DE DADOS E TIPAGEM	8
3. TRATAMENTO DE FALHAS E CONFIABILIDADE	8
4. ENGENHARIA E PERFORMANCE (GENERICS)	9
5. SOLID COMO GUIA PRÁTICO (NÃO ACADÊMICO)	9
6. SINAIS DE ALERTA (BAD SMELLS)	10
RESUMO EM UMA FRASE:	10
GUIA DE ESTUDOS PRÁTICO	11
MÓDULO 1: ENCAPSULAMENTO E INTEGRIDADE DO DOMÍNIO	11
MÓDULO 2: TIPAGEM SEMÂNTICA E VALUE OBJECTS (VO)	11
MÓDULO 3: GENERICS E PERFORMANCE	12
MÓDULO 4: TRATAMENTO DE FALHAS E RESULTADOS	12
MÓDULO 5: SOLID NA PRÁTICA	13
FRASES DE IMPACTO	14
SOBRE FILOSOFIA DE DESIGN E DOMÍNIO	14
SOBRE MATURIDADE E CULTURA DEV	14
SOBRE TRATAMENTO DE ERROS E FALHAS	15
SOBRE SOLID E ARQUITETURA	16
GLOSSÁRIO ABRANGENTE	17
TABELA DE RELACIONAMENTO ONTOLÓGICO	24
MOMENTOS MAIS INTERESSANTES DO ENCONTRO	29
PERGUNTAS E RESPOSTAS DO ENCONTRO	32
AGORA É COM VOCÊ	38
COMPARTILHE SUA JORNADA	38

1. ENCAPSULAMENTO E A INTEGRIDADE DO DOMÍNIO

Francisco inicia combatendo o "padrão" comum de criar classes anêmicas com propriedades públicas e sets abertos.

- O Domínio não deve ser "simpático": Schneider utiliza a analogia de cadeados. Embora o Front-end e as camadas de borda (Handlers/Commands) devam validar os dados (*Fail-Fast*), o Domínio é o último e mais importante cadeado. Ele não deve "confiar" que as camadas externas limpam o dado; ele deve garantir sua própria integridade, impedindo a criação de objetos em estado inválido.
- Expressividade de Negócio: Em vez de expor propriedades para alteração direta, as classes devem expor métodos que representem operações de negócio (ex: em vez de `p.Status = "Casado"`, usar `p.Casar()`). Isso centraliza as regras de transição de estado e torna o código mais testável e semântico.

2. TIPAGEM SEMÂNTICA VS. OBSESSÃO POR PRIMITIVOS

Um ponto central da aula é o uso de Value Objects (VO) para combater a dependência excessiva de strings e ints para representar conceitos complexos (como CPF, CNPJ ou Endereço).

- Implementação Moderna: Francisco recomenda o uso de `readonly struct` ou `record struct` para VOs, aproveitando o "açúcar sintático" do C# para garantir imutabilidade e comparação por valor (não por referência).
- Operadores Implícitos: Uma técnica destacada para lidar com sistemas legados é o uso de operadores de conversão implícita. Isso permite trocar uma string por um objeto `Cpf` em um construtor sem quebrar os consumidores antigos, facilitando a modernização gradual do domínio.

3. HERANÇA, COMPOSIÇÃO E POLIMORFISMO

A discussão vai além das definições acadêmicas:

- Trade-offs da Herança: Schneider alerta que a herança é frequentemente usada de forma errada apenas para reaproveitar código, o que aumenta o

acoplamento e a carga cognitiva. Ele manifesta preferência pela Composição, que tende a ser mais explícita e menos complexa.

- Polimorfismo: É apresentado como a ferramenta para intercâmbio de implementações concretas em tempo de execução, permitindo que o sistema seja extensível sem modificar o orquestrador.

4. GENERICS: PERFORMANCE E ABSTRAÇÃO

O instrutor reforça que Generics não servem apenas para reutilização de código, mas são fundamentais para a performance, pois evitam operações de *boxing* e *unboxing* (muito custosas em escala).

- Variância: Foram explicados os conceitos de Covariância (permitindo tipos mais derivados, comum em `IEnumerable`) e Contravariância (tipos menos derivados, comum em `Action<T>`), enfatizando como o compilador pode ser um aliado na segurança de tipos.

5. TRATAMENTO DE FALHAS: EXCEPTIONS VS. RESULT PATTERN

Este é um dos debates mais ricos da aula, focando em semântica e maturidade do time:

- Exceptions: Devem ser reservadas para cenários verdadeiramente excepcionais (erros de infraestrutura, conexões perdidas). Francisco ensina a forma correta de relançar exceções usando apenas `throw` (para preservar a *Stack Trace* completa) em vez de `throw ex`.
- Result Pattern & Maybe: Para fluxos de negócio onde a falha é um resultado previsto (ex: CPF inválido), recomenda-se o Result Pattern (usando a biblioteca de Vladimir Khorikov). Isso torna a assinatura do método honesta: o chamador sabe, em tempo de compilação, que precisa tratar um sucesso ou uma falha, aumentando a legibilidade e removendo a "mágica" das exceções ocultas.

6. SOLID SOB UMA ÓTICA PRÁTICA

Francisco revisita o SOLID focando em sinais de alerta (*Bad Smells*):

- SRP (Responsabilidade Única): Não é sobre o número de métodos, mas sobre "atores" e razões para mudar. Classes com muitos serviços injetados ou sufixos como Manager/Helper são "ímãs de código" que violam este princípio.
- OCP (Aberto/Fechado): O objetivo é que novas funcionalidades (como um novo meio de pagamento) gerem novos arquivos, não alterações em switches ou ifs existentes. O uso de dicionários de estratégias injetados via DI é a solução sugerida.
- LSP (Liskov): Trata-se de confiança. Se uma classe filha lança NotImplementedException para um método da base, ela quebra o contrato e a previsibilidade do sistema.
- ISP (Segregação de Interface): Interfaces devem ser vistas como papéis (roles). É preferível ter várias interfaces pequenas (ex: IReadRepository, IWriteRepository) do que uma interface "bazuca" que obriga a implementação de métodos desnecessários.
- DIP (Inversão de Dependência): O Domínio deve ser agnóstico a frameworks. Detalhes de implementação (banco de dados, APIs de terceiros) devem depender de abstrações definidas pelo domínio, resolvendo o acoplamento no *Composition Root*.

CONCLUSÃO

A aula de Francisco Schneider é um manifesto contra o código accidental. Ele defende que cada decisão de design — desde o uso de uma struct até a escolha entre uma Exception ou um Result — deve ser intencional, considerando a performance, a legibilidade e a longevidade da aplicação.

INSIGHTS E TAKEAWAYS PRÁTICOS



1. FILOSOFIA DE DESIGN DE DOMÍNIO

- O Domínio não deve ser "Simpático": Este é um dos maiores insights da reunião. Schneider defende que o Domínio é a camada de segurança máxima. Ele não deve confiar nas validações do Front-end ou da API. Se um dado inválido chegou ao Domínio, o sistema deve "reagir mal" (através de exceções ou falhas) para garantir que o estado interno nunca seja corrompido.
- Intencionalidade acima do Acidente: O código "simples" (tudo público e anêmico) muitas vezes é simples por acidente ou preguiça, não por design. O design sênior exige que cada decisão (expor ou não um método) seja intencional.

2. MODELAGEM DE DADOS E TIPAGEM

- Combate à Obsessão por Primitivos: Tratar CPFs, e-mails ou endereços apenas como string é um risco. O takeaway é encapsular esses conceitos em Value Objects (VOs).
- Poder das Structs Modernas: No C# moderno, readonly record struct é a ferramenta ideal para VOs, unindo performance (alocação na Stack), imutabilidade e comparação por valor.
- Modernização do Legado via Operadores Implícitos: Um takeaway técnico valioso: use implicit operator para permitir que o código legado continue enviando string, enquanto o seu novo Domínio já trabalha com objetos ricos. Isso reduz o custo de refatoração.

3. TRATAMENTO DE FALHAS E CONFIABILIDADE

- Assinaturas "Honestas": Métodos que retornam apenas o objeto (ex: GetByld) mentem, pois podem retornar nulo ou lançar exceção. O uso do Result Pattern torna a assinatura honesta: o desenvolvedor vê que o retorno pode ser uma falha e é obrigado a tratar isso em tempo de compilação.

- A Anatomia da Exceção: Nunca use `throw ex;`, pois você destrói a rastreabilidade (*stack trace*). Use apenas `throw`; As exceções devem ser ricas e semânticas, criadas apenas quando um tratamento diferenciado for necessário.
- Maybe/Optional para Semântica: Usar `Maybe<T>` é superior a retornar nulo, pois deixa claro para o consumidor que a ausência de valor é um cenário esperado de negócio (ex: busca por um critério que pode não existir).

4. ENGENHARIA E PERFORMANCE (GENERICIS)

- Generics além do Reuso: O uso de tipos genéricos é uma estratégia de performance. Ele evita o *boxing* (converter tipos de valor para referência) e o *unboxing*, que são extremamente custosos em aplicações de alta escala.
- Segurança via Restrições: O uso de `where T : new()` ou `where T : notnull` permite que o compilador ajude a evitar erros que só apareceriam em produção (*runtime*).

5. SOLID COMO GUIA PRÁTICO (NÃO ACADÊMICO)

- Interfaces são Papéis (Roles): Não crie interfaces que são espelhos de suas classes (ex: `ICustomerService` com 20 métodos). Aplique o ISP para criar interfaces baseadas no que o cliente precisa (ex: `ICustomerReader`).
- Blast Radius (Raio de Explosão): O SRP (Responsabilidade Única) visa diminuir o impacto de uma mudança. Se alterar uma regra de negócio quebra o banco de dados, sua classe tem "interessados" demais.
- Injeção de Dependência e Acoplamento: O takeaway é: o Domínio define o contrato (interface), e a Infraestrutura o implementa. O acoplamento deve ser resolvido apenas no Composition Root (geralmente no `Program.cs`), mantendo o núcleo do software puro e agnóstico a ferramentas.

6. SINAIS DE ALERTA (BAD SMELLS)

- Imãs de Código: Cuidado com classes com sufixos `Manager`, `Helper` ou `Service` que crescem indefinidamente. Elas são sinais clássicos de violação do SRP.

- Excesso de Mocks em Testes: Se para testar um método você precisa "mockar" 10 dependências, seu código está excessivamente acoplado e violando os princípios de coesão do SOLID.

RESUMO EM UMA FRASE:

"O código deve ser projetado para que o desenvolvedor tenha que fazer um esforço extra para cometer um erro, usando o compilador e o design de tipos como aliados."

GUIA DE ESTUDOS PRÁTICO



MÓDULO 1: ENCAPSULAMENTO E INTEGRIDADE DO DOMÍNIO

1. Por que Schneider afirma que o domínio não deve ser "simpático"?

- Resposta: Porque o Domínio é a camada de segurança máxima da aplicação. Ele não deve "confiar" que as camadas externas (Front-end ou Handlers) já validaram os dados. Ele deve ser rigoroso e impedir que qualquer objeto seja instanciado em um estado inválido, sendo o "último cadeado" da integridade.

2. Qual o risco de manter set público em propriedades de classes de negócio?

- Resposta: Isso cria "Domínios Anêmicos". Qualquer parte do sistema pode alterar o estado do objeto de forma descontrolada, pulverizando regras de negócio por toda a aplicação e dificultando a manutenção e o rastreamento de bugs.

3. Como substituir propriedades abertas por operações de negócio expressivas?

- Resposta: Deve-se fechar os sets (usando private set ou init) e expor métodos que representem ações reais do mundo físico (ex: em vez de mudar a data de um contrato manualmente, usar um método RenovarContrato(DateTime novaData) que contenha as validações necessárias).

MÓDULO 2: TIPAGEM SEMÂNTICA E VALUE OBJECTS (VO)

4. O que é "Obsessão por Primitivos" e como combatê-la?

- Resposta: É o hábito de usar tipos básicos (string, int) para representar conceitos complexos (CPF, E-mail, CEP). O combate é feito criando Value Objects, que são tipos customizados que encapsulam o valor e a lógica de validação daquele dado.

5. Qual a recomendação de implementação moderna para VOs em C#?

- Resposta: Utilizar readonly record struct. Elas oferecem imutabilidade, comparação por valor (e não por referência) e performance superior por serem alocadas na *Stack* e evitarem o custo do *Garbage Collector*.

6. Como o implicit operator ajuda na refatoração de sistemas legados?

- Resposta: Ele permite que o compilador converta automaticamente um tipo (como string) para um VO (como Cpf). Isso permite atualizar o domínio para usar tipos ricos sem precisar alterar centenas de chamadas de métodos que ainda enviam strings simples.

MÓDULO 3: GENERICS E PERFORMANCE

7. Além do reuso de código, qual a grande vantagem técnica do Generics?

- Resposta: Performance. Generics evitam o Boxing (converter tipo de valor para referência) e o Unboxing, operações que consomem muita memória e CPU em aplicações de alta escala.

8. Explique a diferença prática entre Covariância e Contravariância.

- Resposta:
 - Covariância (out T): Permite usar um tipo mais derivado do que o originalmente especificado (ex: passar uma List<Manager> para um método que espera IEnumerable<Employee>). É comum em retornos.
 - Contravariância (in T): Permite usar um tipo menos derivado (mais genérico) do que o especificado (ex: passar uma Action<Employee> para onde se espera uma Action<Manager>). É comum em entradas/parâmetros.

MÓDULO 4: TRATAMENTO DE FALHAS E RESULTADOS

9. Qual a diferença entre throw; e throw ex;?

- Resposta: throw; relança a exceção preservando toda a *Stack Trace* original (o caminho exato do erro). throw ex; "reseta" a exceção, fazendo parecer que o erro começou naquele ponto, o que destrói a capacidade de *troubleshooting*.

10. Quando devo usar o Result Pattern em vez de lançar uma Exception?

- Resposta: Use Result Pattern para falhas que fazem parte do fluxo esperado do negócio (ex: "Saldo Insuficiente", "CPF Inválido"). Use Exceptions apenas para situações catastróficas ou imprevistas (ex: "Banco de dados fora do ar", "Disco cheio").

11. Qual a vantagem de usar o tipo Maybe<T>?

- Resposta: Ele elimina a ambiguidade do null. Ao retornar Maybe<Professional>, você diz explicitamente ao chamador que o profissional pode não ser encontrado e o obriga a tratar esse cenário (usando propriedades como HasValue), tornando o código "honesto".

MÓDULO 5: SOLID NA PRÁTICA

12. Como identificar uma violação do Princípio de Responsabilidade Única (SRP)?

- Resposta: Observe o "Blast Radius" (raio de explosão). Se uma alteração em uma regra de cálculo quebra o envio de e-mails, a classe tem responsabilidades demais. Outro sinal é o excesso de serviços injetados no construtor.

13. No Princípio Aberto/Fechado (OCP), como evitar o uso de switches infinitos?

- Resposta: Utilizando o padrão Strategy aliado à Injeção de Dependência. Em vez de um switch para decidir o tipo de pagamento, injeta-se uma coleção de implementações (IEnumerable<IPaymentProcessor>) e utiliza-se um dicionário para selecionar a correta em tempo de execução.

14. Por que interfaces não devem ser "espelhos" de classes concretas (ISP)?

- Resposta: Porque isso viola o Princípio de Segregação de Interface. Interfaces devem representar Papéis (Roles). Um cliente que só precisa ler dados não deve depender de uma interface que o obriga a conhecer métodos de exclusão ou atualização.

FRASES DE IMPACTO



SOBRE FILOSOFIA DE DESIGN E DOMÍNIO

- "O domínio não pode ser simpático."
 - Contexto: Ao discutir validações, Francisco defende que o Domínio é a última linha de defesa. Ele não deve "confiar" ou ser gentil com dados vindos de fora; deve ser rigoroso e falhar imediatamente se houver tentativa de criar um objeto inválido.
- "Não deixe sua casa toda aberta, é inseguro."
 - Contexto: Uma analogia sobre encapsulamento. Schneider critica o hábito de deixar todas as propriedades como public get/set, defendendo que devemos expor apenas o mínimo necessário para o mundo externo.
- "É muito mais fácil tu ir abrindo para o mundo do que o contrário."
 - Contexto: Conselho prático de design. Comece com tudo privado e fechado; é fácil tornar algo público depois, mas é quase impossível fechar algo que já tem centenas de referências no sistema sem quebrar o código.
- "O código deve ser projetado para que o desenvolvedor tenha que fazer um esforço extra para fazer 'caquinha!'"
 - Contexto: Sobre usar o compilador e o design de tipos (como Value Objects) para impedir erros humanos comuns, tornando o "caminho certo" o mais fácil de seguir.

SOBRE MATURIDADE E CULTURA DEV

- "Um dev sênior deve, no mínimo, saber a diferença de quando utilizar struct e quando utilizar classe."
 - Contexto: Francisco cita Elemar Jr. para enfatizar que o conhecimento dos fundamentos da linguagem (.NET) e do gerenciamento de memória (Stack vs. Heap) é o que diferencia profissionais experientes.

- "Nós, como devs, adoramos reaproveitar código... e isso é usado para o mal diversas vezes."
 - Contexto: Um alerta sobre o uso indiscriminado de Herança e interfaces gigantescas. Muitas vezes, a busca cega pelo reuso cria acoplamento excessivo e código difícil de ler.
- "A dose é que faz o veneno."
 - Contexto: Referindo-se ao uso de técnicas de Programação Funcional (como mônades e encadeamentos) em C#. Em excesso, tornam o código ilegível; em equilíbrio, trazem elegância.
- "Onde há fumaça, há fogo."
 - Contexto: Sobre os *Bad Smells* (maus cheiros) no código. Sinais como classes com centenas de linhas ou excesso de injeções de dependência indicam quase certamente um problema de design oculto.

SOBRE TRATAMENTO DE ERROS E FALHAS

- "Não existe almoço grátis: Exception é extremamente rica em informações, consequentemente ela é pesada."
 - Contexto: Explicando o trade-off de performance das exceções. Elas são ótimas para diagnóstico, mas o custo computacional existe e deve ser considerado em cenários de alta escala.
- "O Result Pattern torna a assinatura do método honesta."
 - Contexto: Ao comparar Exceções com o Result Pattern, Francisco argumenta que o Result deixa claro no contrato do método que uma falha pode ocorrer, enquanto a Exceção é uma "surpresa" em tempo de execução.
- "Throw ex destrói a rastreabilidade; use apenas throw."
 - Contexto: Um ensinamento técnico crítico sobre preservação da *Stack Trace* original em C#. Relançar uma exceção de forma errada apaga o histórico de onde o erro realmente começou.

SOBRE SOLID E ARQUITETURA

- "Interfaces deveriam ser como 'roles' (papéis): elas são habilitadoras, não espelhos das classes."
 - Contexto: Crítica ao hábito de criar uma interface para cada classe (IService -> Service). Interfaces devem representar comportamentos específicos que o cliente precisa, seguindo o Princípio de Segregação de Interface (ISP).
- "Liskov trata-se de confiança e previsibilidade."
 - Contexto: Simplificação do Princípio de Substituição de Liskov. Se você não pode confiar que uma classe filha se comporta como a pai sem precisar de um if para checar o tipo, o design está quebrado.
- "Classes com sufixos Manager, Helper ou Service são 'ímãs de código'. "
 - Contexto: Alerta sobre o Princípio de Responsabilidade Única (SRP). Essas nomenclaturas genéricas convidam os desenvolvedores a empilhar funções desconexas no mesmo arquivo até que ele se torne imantável.
- "Abstrações não devem depender de detalhes; são os detalhes que dependem de abstrações."
 - Contexto: A essência da Inversão de Dependência (DIP). O domínio (regra de negócio) define o que precisa, e a infraestrutura (banco de dados, APIs) apenas obedece.

GLOSSÁRIO ABRANGENTE



Elemento	Descrição do Conceito
Açúcar Sintático (Syntactic Sugar)	Recursos de uma linguagem de programação (como o record no C#) que facilitam a escrita de código mais legível e conciso, sem alterar a funcionalidade técnica subjacente.
Blast Radius (Raio de Explosão)	Termo usado para descrever a amplitude do impacto que uma alteração em um trecho de código causa no restante do sistema. Um bom design (SRP) busca minimizar esse raio.
Boxing e Unboxing	Processo de converter um tipo de valor (como int ou struct) para um tipo de referência (object) e vice-versa. É uma operação custosa para a performance e memória, evitada pelo uso de <i>Generics</i> .
CA-2201	Diretriz de análise de código da Microsoft que estabelece boas práticas sobre quais tipos de exceções não devem ser lançadas manualmente (como <code>NullReferenceException</code>).
Camada Anticorrupção (ACL)	Padrão de design que isola o domínio principal de influências externas (como APIs de terceiros ou bancos de

Elemento	Descrição do Conceito
	dados), garantindo que o núcleo do sistema permaneça puro e agnóstico.
Composition Root	O ponto único de entrada de uma aplicação (geralmente o Program.cs) onde todas as dependências e abstrações são resolvidas e configuradas via Injeção de Dependência.
Composição	Técnica de design que favorece a criação de objetos complexos através da junção de objetos simples (relação "tem um"), sendo geralmente preferível à herança por reduzir o acoplamento.
Contravariância (in T)	Recurso de <i>Generics</i> que permite utilizar um tipo menos derivado (mais genérico) do que o originalmente especificado. É comum em entradas de métodos e delegados como Action<T>.
Covariância (out T)	Recurso de <i>Generics</i> que permite utilizar um tipo mais derivado (mais específico) do que o originalmente especificado. Comumente visto em interfaces de leitura como IEnumerable<T>.

Elemento	Descrição do Conceito
DIP (Dependency Inversion)	Princípio que dita que módulos de alto nível (domínio) não devem depender de módulos de baixo nível (infraestrutura), mas ambos devem depender de abstrações (interfaces).
Domínio Anêmico	Modelo de design onde as classes de negócio são apenas "sacos de dados" (propriedades públicas), sem comportamento ou lógica, o que leva ao vazamento de regras de negócio para camadas externas.
Encapsulamento	Pilar da Orientação a Objetos que consiste em esconder os detalhes internos de funcionamento de um objeto e proteger seu estado, expondo apenas operações seguras e intencionais.
EximiaCo	Consultoria técnica especializada em arquitetura e engenharia de software onde o conteúdo discutido é fundamentado.
Fail-Fast	Filosofia de design que prega a validação de dados o mais cedo possível no fluxo de execução, interrompendo o processo

Elemento	Descrição do Conceito
	imediatamente ao detectar um erro para evitar efeitos colaterais.
ISP (Interface Segregation)	Princípio que defende que uma classe não deve ser forçada a depender de métodos que ela não utiliza. Propõe a divisão de interfaces grandes em interfaces menores e específicas (papéis).
LSP (Liskov Substitution)	Princípio que garante que uma classe filha deve poder substituir sua classe pai sem alterar a correção ou o comportamento esperado do sistema. Baseia-se em confiança e previsibilidade.
Maybe Pattern (Optional)	Tipo de retorno que substitui o uso do null. Ele força o desenvolvedor a lidar explicitamente com o cenário em que um valor pode estar ausente, aumentando a segurança do código.
Mônades (Monads)	Conceito da programação funcional aplicado ao C# (especialmente no <i>Result Pattern</i>) para encadear operações de forma que o fluxo de erro seja propagado sem a necessidade de múltiplos blocos if.

Elemento	Descrição do Conceito
Obsessão por Primitivos	<i>Code Smell</i> que ocorre quando tipos básicos (string, int, double) são usados excessivamente para representar conceitos de negócio que deveriam ser objetos ricos (como CPF ou Moeda).
OCP (Open/Closed Principle)	Princípio que estabelece que entidades de software devem estar abertas para extensão (novas funcionalidades), mas fechadas para modificação (alterar o código núcleo existente).
Operador Implícito	Recurso do C# que permite a conversão automática entre tipos (ex: converter uma string diretamente em um objeto Cpf), útil para refatorar sistemas legados sem quebrar contratos.
Readonly Record Struct	Estrutura moderna no .NET ideal para implementar <i>Value Objects</i> , pois combina imutabilidade, performance (alocação na Stack) e comparação por valor de forma nativa.
Result Pattern	Padrão que utiliza um objeto para envolver o retorno de uma operação, indicando explicitamente se houve

Elemento	Descrição do Conceito
	sucesso ou falha, em vez de depender de exceções para controle de fluxo.
SharpLab	Ferramenta online utilizada para inspecionar como o compilador do C# transforma o código em IL (<i>Intermediate Language</i>) ou em versões descompiladas de baixo nível.
SRP (Single Responsibility)	Princípio que define que uma classe deve ter apenas uma razão para mudar, ou seja, deve servir a apenas um ator ou interessado no sistema.
Stack vs Heap	Áreas de memória do computador. A <i>Stack</i> é rápida e usada para tipos de valor e structs; a <i>Heap</i> é usada para objetos de classe e gerenciada pelo <i>Garbage Collector</i> .
Stack Trace	Relatório que mostra a pilha de chamadas de métodos ativa em um determinado momento, essencial para identificar a origem exata de uma exceção durante o <i>troubleshooting</i> .
Value Object (VO)	Objeto que não possui identidade única (ID) e é definido exclusivamente pelo valor de suas propriedades. Dois

Elemento	Descrição do Conceito
	VOs são considerados iguais se seus valores forem idênticos.
Vladimir Khorikov	Autor técnico de referência citado indiretamente pela biblioteca <i>CSharpFunctionalExtensions</i> , focada em Result Pattern e domínio rico.

TABELA DE RELACIONAMENTO ONTOLÓGICO



Conceito A	Conceito B	Detalhamento da Relação
Encapsulamento	Integridade do Domínio	O encapsulamento é o mecanismo técnico utilizado para proteger e garantir a integridade do domínio, fechando o estado interno e expondo apenas operações intencionais.
Value Objects (VO)	Obsessão por Primitivos	Value Objects são a solução direta para a Obsessão por Primitivos, substituindo tipos genéricos (como strings) por tipos ricos que carregam significado de negócio e validação.
Generics	Performance	O uso de tipos genéricos melhora a performance do sistema ao eliminar a necessidade de <i>boxing</i> e <i>unboxing</i> , reduzindo a pressão sobre o gerenciador de memória (<i>Garbage Collector</i>).

Conceito A	Conceito B	Detalhamento da Relação
Result Pattern	Assinaturas Honestas	O Result Pattern torna as assinaturas dos métodos "honestas", pois obriga o contrato a declarar explicitamente que uma operação pode falhar, em vez de esconder falhas atrás de exceções.
SRP (Responsabilidade Única)	Blast Radius (Raio de Explosão)	O princípio SRP visa minimizar o <i>Blast Radius</i> de uma mudança; ao isolar uma única razão para mudar, evita-se que alterações em um módulo causem efeitos colaterais em áreas desconexas.
LSP (Liskov Substitution)	Confiança e Previsibilidade	O princípio de Liskov estabelece a base da confiança técnica: ele garante que o sistema se comporte de forma previsível independentemente da implementação concreta utilizada.

Conceito A	Conceito B	Detalhamento da Relação
DIP (Inversão de Dependência)	Composition Root	O DIP é o princípio de design (o "quê"), enquanto o <i>Composition Root</i> é o local físico na arquitetura (o "onde") onde esse princípio é materializado através da Injeção de Dependência.
Implicit Operators	Refatoração de Legado	Operadores implícitos servem como uma ponte de compatibilidade, permitindo que um Domínio Moderno (com VOs) aceite dados de um sistema legado (com strings) sem quebras imediatas.
ISP (Segregação de Interface)	Papéis (Roles)	O ISP transforma interfaces "bazuca" em papéis específicos (<i>Roles</i>), garantindo que um cliente dependa apenas do comportamento estrito que ele necessita executar.

Conceito A	Conceito B	Detalhamento da Relação
Maybe Pattern	Nulo Implícito	O padrão <i>Maybe</i> substitui o perigo do nulo implícito por um tipo explícito, forçando o desenvolvedor a tratar a ausência de valor como um cenário de negócio válido.
ReadOnly Record Struct	Stack Memory	Estruturas do tipo <i>readonly record struct</i> são otimizadas para serem alocadas na <i>Stack</i> , o que garante imutabilidade por design e ganho de velocidade em relação à <i>Heap</i> .
Fail-Fast	Camadas de Borda	A filosofia <i>Fail-Fast</i> deve ser aplicada nas Camadas de Borda (como <i>Commands</i> e <i>Handlers</i>) para impedir que dados sabidamente inválidos consumam recursos processando até o núcleo do sistema.
Exceções Customizadas	Semântica de Tratamento	A criação de uma nova exceção só se justifica

Conceito A	Conceito B	Detalhamento da Relação
		ontologicamente se ela exigir uma semântica de tratamento (um bloco <i>catch</i> específico) diferente das exceções já existentes.
Domínio Rico	Mantenabilidade	Um Domínio Rico (com lógica e proteção de estado) aumenta a manutenibilidade a longo prazo, pois centraliza o conhecimento de negócio em um único local seguro e testável.

MOMENTOS MAIS INTERESSANTES DO ENCONTRO



Título	O que foi dito	Justificativa
A "Hostilidade" Necessária do Domínio	O domínio não pode ser "simpático"; ele deve ser o último cadeado e reagir mal a dados inválidos.	Este é o ponto de virada da aula. Schneider quebra a ideia de que validação é apenas para "ajudar o usuário" (UX) e a coloca como um pilar de segurança sistêmica (Integridade).
O "Pulo do Gato" para Sistemas Legados	O uso de implicit operator para converter string em <i>Value Objects</i> sem quebrar o código antigo.	Demonstra senioridade prática. Em vez de propor uma refatoração impossível de meses, ele oferece uma solução técnica que permite modernizar o núcleo do sistema mantendo a retrocompatibilidade.
A Morte do throw ex;	A explicação técnica de por que usar apenas throw; para preservar a <i>Stack Trace</i> original.	Resolve um dos erros mais comuns e catastróficos em produção: a perda do rastro do erro. É um detalhe de "baixo nível" que separa o desenvolvedor que

Título	O que foi dito	Justificativa
		entende a CLR do que apenas escreve código.
Interfaces como Papéis (Roles), não Espelhos	A crítica ao hábito de criar interfaces que apenas replicam os métodos da classe concreta (IService -> Service).	Ataca um "vício de design" amplamente disseminado. Schneider força os alunos a pensarem na interface sob a ótica do consumidor (o que ele precisa acessar) e não do provedor (o que a classe tem).
O Conceito de "Imã de Código"	Como sufixos genéricos (Manager, Helper) convidam o desenvolvedor a violar o SRP (Responsabilidade Única).	É uma análise sociológica do código. O nome de uma classe dita o comportamento dos desenvolvedores; nomes ruins atraem código ruim. Identificar isso é crucial para evitar a degradação arquitetural.
Arquitetura é sobre Pessoas (Maturidade do Time)	A decisão entre usar <i>Exception</i> ou <i>Result Pattern</i> deve considerar a senioridade e a cultura da equipe.	Schneider demonstra que arquitetura não é uma ciência exata feita no vácuo. Um design "perfeito" que o time não sabe manter é, na verdade, um design ruim. É o

Título	O que foi dito	Justificativa
		reconhecimento do fator humano na engenharia.
Performance Silenciosa com Generics	O uso de Generics para evitar o <i>Boxing</i> e <i>Unboxing</i> em aplicações de alta escala.	Eleva a discussão de "reuso de código" para "eficiência computacional". Mostra que o bom uso de Orientação a Objetos tem impacto direto no custo de infraestrutura e latência da aplicação.

PERGUNTAS E RESPOSTAS DO ENCONTRO

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
Márcio	Existe previsão para novas turmas online do curso "DDD do Jeito Certo"?	Francisco informa que as turmas são sazonais e pede para a equipe da EximiaCo postar informações detalhadas no grupo de WhatsApp.	Satisfatória. Como o foco da aula era técnico, Francisco agiu corretamente ao não desviar o assunto, delegando a resposta comercial ao suporte.
Gustavo	Sobre a classe Person, por que a implementação com métodos setName é melhor?	Ele argumenta que isso encapsula o ponto de alteração e permite garantir a validade do dado antes de mudar o estado.	Satisfatória. Francisco complementa a visão de Gustavo explicando que, embora o C# tenha "açúcar sintático" para propriedades, o foco deve ser em métodos que representem operações de negócio.

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
Farlei	Se estou modelando uma pessoa, o comportamento de negócio (como mudar estado civil) deveria estar fora da classe?	Francisco explica que operações significativas (ex: Casar()) devem estar dentro da classe para proteger as regras e o estado, em vez de apenas expor propriedades.	Satisfatória. A resposta reforça o conceito de "Domínio Rico" contra o "Domínio Anêmico", ponto central da aula.
Farlei	É uma boa prática usar o <i>Builder Pattern</i> para fazer as validações de negócio e deixar a classe apenas como modelo?	O <i>Builder</i> é um padrão criacional (para criar o objeto). Para alterações de estado após o objeto já existir, a regra deve estar em métodos na classe de domínio.	Altamente Satisfatória. Francisco desfaz uma confusão comum entre padrões de criação e padrões de comportamento/e stado.
Gustavo	O uso de <i>Records</i> e <i>Structs</i> ainda sofre resistência por falta de conhecimento dos desenvolvedores?	Francisco concorda que é uma questão de cultura e "preguiça" técnica, mas que um dev sênior precisa dominar	Satisfatória. A resposta serviu como um "choque de realidade" sobre a importância de conhecer a fundo a linguagem e o

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
		esses fundamentos para garantir performance.	gerenciamento de memória.
Lucas	No uso de Generics, a restrição new() significa que a classe deve ser instanciável?	Francisco ajusta o termo: ela obriga a existência de um construtor padrão (sem parâmetros), o que é essencial para alguns frameworks e ORMs.	Satisfatória. Precisão técnica importante para evitar erros de compilação em implementações genéricas.
Gustavo	É válido criar um tipo DomainException para tratar erros de negócio?	Sim. Ele inclusive sugere o uso de filtros na API para converter esses tipos de exceção automaticamente em Status Codes HTTP (como 422 ou 400).	Satisfatória. Francisco transformou uma dúvida de design em uma solução arquitetural prática e elegante.
Farlei	Devo criar exceções específicas, como	Francisco orienta que você só deve criar um novo tipo de exceção se ela	Satisfatória. A resposta evita o "over-engineering" (excesso de

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
	uma <code>CnpjException</code> ?	exigir um tratamento (um bloco <i>catch</i>) diferenciado.	engenharia) de criar centenas de tipos de exceção sem utilidade prática.
Farlei	Colocar validações em várias camadas (Front, Handler, Domínio) não gera duplicação de código?	Você pode validar em vários locais (proteção), mas a lógica de escrita não precisa ser duplicada se você encapsular a regra no <i>Value Object</i> (ex: no VO de CPF).	Altamente Satisfatória. Resolve o paradoxo entre a necessidade de segurança (cadeados) e a boa prática de não repetir código (DRY).
Jordano	Posso injetar o validador do <i>FluentValidation</i> direto no construtor da entidade de domínio?	Não é recomendado. O domínio deve ser agnóstico a frameworks. É melhor usar o <i>Result Pattern</i> ou validações manuais para manter o domínio puro.	Satisfatória. Protege a arquitetura contra o acoplamento com bibliotecas externas no núcleo do sistema.

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
Márcio	Se eu lançar uma exceção no domínio, não perco a capacidade de retornar todos os erros de um formulário de uma vez?	Francisco concorda. As exceções no domínio são o último recurso. Para coletar múltiplos erros de validação, o <i>Result Pattern</i> ou validações prévias no <i>Command/Handler</i> são melhores.	Satisfatória. Reconhece que para UX (experiência do usuário), as exceções secas não são a melhor ferramenta.
Eduardo	Como medir se uma exceção customizada é muito custosa para a performance?	Através de testes de carga, estresse e ferramentas de <i>benchmark</i> . Mas Francisco ressalta que a decisão deve ser primeiro semântica e depois de performance.	Satisfatória. Ele retira o peso da "paralisia por análise" de performance, focando na clareza do código.
Eduardo	O padrão <i>Maybe</i> é uma abstração do <i>Result Pattern</i> ou do <i>Null Object Pattern</i> ?	Ele é um complemento. Serve para dar semântica a casos onde a ausência	Satisfatória. Clarifica a diferença sutil entre "deu erro" (<i>Result Failure</i>) e

Participante	Pergunta / Questionamento	Resposta de Francisco Schneider	Análise da Resposta
		de valor é um cenário esperado, não uma falha (ex: buscar algo que pode ou não existir).	"não encontrei nada" (Maybe None).
Farlei	O padrão <i>Mediator</i> ajuda a implementar o Princípio Aberto/Fechado (OCP)?	O <i>Mediator</i> desacopla quem chama de quem executa, mas o padrão que realmente brilha no OCP é o <i>Decorator</i> ou o uso de dicionários de estratégia.	Satisfatória. Francisco faz uma distinção importante entre "desacoplamento" e "extensibilidade sem modificação".

AGORA É COM VOCÊ



Este resumo foi elaborado para apoiar a consolidação dos principais conceitos discutidos na sessão. No entanto, o verdadeiro valor deste workshop está em transformar conhecimento em prática.

- Aplique o que foi discutido
- Reflita sobre os insights e leve-os para seu contexto profissional
- Experimente, teste abordagens, aprenda com os resultados
- Adapte os aprendizados à realidade da sua equipe e dos seus projetos
- Evolua um pouco mais a cada sessão
- Quem pratica com intenção evolui com consistência.

COMPARTILHE SUA JORNADA

Use o LinkedIn (ou outra rede profissional) para dividir reflexões, descobertas e experiências. Isso contribui para o seu posicionamento como alguém em constante evolução.

MARQUE A @EXIMIA.CO

ele acompanha e interage com a comunidade de mentorandos.



PARTICIPE DO GRUPO FECHADO NO WHATSAPP

O grupo é seu canal direto com os instrutores e com outros participantes do Workshop em C# do Jeito Certo.

Utilize-o para esclarecer dúvidas, pedir opiniões, compartilhar resultados e fortalecer sua rede.

CLUBE DE ESTUDOS



elemarjr.com